



# OASIS Security Report

## Insecure Direct Object Reference Security Analysis

**Date:** 2026-06-12 00:12:54 — **Model:** llama3.2:3b

**Vulnerability:** Insecure Direct Object Reference

**Project:** vulnerable

**Analysis root:** `../vulnerable`

### Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

## Summary

Analyzed **9** file(s) for Insecure Direct Object Reference.

[Return to table of contents](#)

## Statistics

**Files analyzed:** 9    **Total findings:** 15

Critical



5

|        |             |   |
|--------|-------------|---|
| High   | <div></div> | 8 |
| Medium | <div></div> | 2 |
| Low    | <div></div> | 0 |

[Return to table of contents](#)

## Files

| File                                                                                           | Similarity |
|------------------------------------------------------------------------------------------------|------------|
| /work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol                          | 0.718      |
| /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol                        | 0.682      |
| /work/project/dataset/vulnerable/reentrancy/simple_dao.sol                                     | 0.678      |
| /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol                        | 0.659      |
| /work/project/dataset/vulnerable/reentrancy/etherstore.sol                                     | 0.654      |
| /work/project/dataset/vulnerable/access-control/mycontract.sol                                 | 0.648      |
| /work/project/dataset/vulnerable/reentrancy/<br>0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol | 0.645      |
| /work/project/dataset/vulnerable/ordering-attacks/<br>eth_tx_order_dependence_minimal.sol      | 0.609      |
| /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol                           | 0.581      |

[Return to table of contents](#)

## Detailed analysis

### File 1: /work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol

Similarity score: 0.718

Source lines: 1-70

#### ► Finding 1.1: Insecure Direct Object Reference (Medium)

##### Vulnerable code

```
function borrow(uint256 borrowAmount) external {  
    // ...  
    deposits[msg.sender] += amount;  
}  
  
function calculateDepositOfWETHRequired(uint256 tokenAmount) public view  
returns (uint256) {  
    return _getOracleQuote(tokenAmount) * depositFactor / 1 ether;  
}
```

##### Explanation

The `deposits` mapping directly references the user's address, allowing an attacker to manipulate it and gain unauthorized access.

##### Impact

Unauthorized access to deposits mapping

##### Entry point

/borrow(uint256)

##### Execution path

```
borrow -> calculateDepositOfWETHRequired -> deposits[msg.sender] += amount;  
deposit -> borrow
```

##### Parameters

msg.sender

##### Exploitation steps

- Manipulate the `deposits` mapping to gain unauthorized access

##### Conditions

User has write access to the contract

**Remediation**

Use an indirect reference to the deposits mapping, such as a unique identifier or a derived value.

## File 2: /work/project/dataset/vulnerable/access-control/parity\_wallet\_bug\_1.sol

Similarity score: 0.682

Source lines: 1-281

### ► Finding 2.1: Insecure Direct Object Reference (Critical)

#### Vulnerable code

```
function revoke(bytes32 _operation) external { ... }
```

#### Explanation

The function 'revoke' does not validate its input parameter '\_operation', making it vulnerable to direct object reference attacks.

#### Impact

Unauthorized access to wallet data

#### Entry point

/Wallet/Abi/revoke

#### Execution path

```
Revoke -> Get Owner Index -> Update Pending State -> Emit Revoke Event
```

#### Parameters

\_operation

#### Exploitation steps

- Send a fake operation hash to the 'revoke' function

#### Example payloads

- 0x1234567890abcdef

#### Conditions

The '\_operation' parameter is not validated.

#### Remediation

Validate the input parameter '\_operation' before passing it to the 'revoke' function

## Secure example

```
function revoke(bytes32 _operation) external { require(msg.sender ==
ownerOf(_operation)); ... }
```

Source lines: 27–358

### ► Finding 2.1: Vulnerability found (High)

#### Vulnerable code

```
function revoke(bytes32 _operation) external { ... }
```

#### Explanation

Direct object reference vulnerability: `\_operation` is directly referenced without proper validation or access control checks.

#### Impact

Unauthorized access to operation data

#### Entry point

/revoke

#### HTTP methods

POST

#### Parameters

\_operation

#### Exploitation steps

- Send a request with a malicious \_operation value

#### Example payloads

- 

#### Conditions

User has permission to revoke operations.

#### Remediation

Use indirect references or validation checks for operation parameters

#### Secure example

```
function revoke(bytes32 _hash) external { ... }
```

Source lines: 72–421

## ► Finding 2.1: Insecure Direct Object Reference (High)

### Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
    // ...
}
```

### Explanation

The `\_to`, `\_value`, and `\_data` parameters are directly referenced in the function call without proper validation or sanitization.

### Impact

Unauthorized access to data, privilege escalation, or data theft

### Entry point

execute(address \_to, uint \_value, bytes \_data)

**Source lines:** 126-467

## ► Finding 2.1: Vulnerability found (Medium)

### Vulnerable code

```
function getOwner(uint ownerIndex) external constant returns (address) {
    return address(m_owners[ownerIndex + 1]);
}
```

### Explanation

Direct object reference in `getOwner` function allows an attacker to access any owner without proper authorization.

### Impact

An attacker can potentially impersonate a legitimate owner and perform unauthorized actions.

### Entry point

/getOwner(uint)

### Execution path

```
GET /getOwner(ownerIndex) -> m_owners[ownerIndex + 1] -> address()
```

### Parameters

ownerIndex

## Exploitation steps

- Find a way to manipulate `ownerIndex`
- Use the manipulated value to access any owner

## Example payloads

- 0

## Conditions

Access to `m\_owners` array without proper authorization.

## Remediation

Validate and sanitize user input before using it as an index into `m\_owners`

## Secure example

```
uint ownerIndex = uint(msg.sender);
return address(m_owners[ownerIndex + 1]);
```

Source lines: 207-477

### ► Finding 2.1: Insecure Direct Object Reference (High)

#### Vulnerable code

```
uint ownerIndex = m_ownerIndex[uint(msg.sender)];
return !(pending.ownersDone & ownerIndexBit == 0);
```

#### Explanation

The use of `m\_ownerIndex[uint(msg.sender)]` without proper validation allows an attacker to manipulate the owner index and access unauthorized data.

#### Impact

Unauthorized access to data or privilege escalation

#### Entry point

execute

#### Execution path

```
execute -> confirmAndCheck -> m_ownerIndex[uint(msg.sender)]
execute -> hasConfirmed
confirmAndCheck
```

#### Parameters

msg.sender

## Exploitation steps

- Manipulate the owner index to access unauthorized data

## Example payloads

- `attacker controlled msg.sender index`

## Conditions

Owner index is directly accessible

## Remediation

Validate and sanitize user input before accessing owner indices.

## Secure example

```
uint ownerIndex = uint(msg.sender) ? 0 : m_ownerIndex[uint(msg.sender)];  
return !(pending.ownersDone & ownerIndexBit == 0);
```

## File 3: /work/project/dataset/vulnerable/reentrancy/simple\_dao.sol

Similarity score: 0.678

Source lines: 1-35

### ► Finding 3.1: Insecure Direct Object Reference (High)

#### Vulnerable code

```
function withdraw(uint amount) {  
    if (credit[msg.sender] >= amount) {  
        bool res = msg.sender.call.value(amount)();  
        credit[msg.sender] -= amount;  
    }  
}
```

#### Explanation

The contract uses `msg.sender.call.value(amount)()` without proper access control checks, allowing an attacker to manipulate the `to` parameter and gain unauthorized access.

#### Impact

Unauthorized transfer of funds

#### Entry point

/withdraw

#### Parameters

amount

#### Exploitation steps

- An attacker can manipulate the `to` parameter in the `withdraw` function call to steal funds.

#### Example payloads

- `malicious_address`

#### Conditions

Authenticated user with high balance.

#### Remediation

Use a validated ID or a whitelist of approved addresses for transfers.

## Secure example

```
function withdraw(uint amount, address to) {  
    require(to != address(0), "Invalid recipient address");  
    if (credit[msg.sender] >= amount) {  
        credit[to] += amount;  
    }  
}
```

## File 4: /work/project/dataset/vulnerable/access-control/parity\_wallet\_bug\_2.sol

Similarity score: 0.659

Source lines: 1-227

### ► Finding 4.1: Insecure Direct Object Reference (Critical)

#### Vulnerable code

```
function revoke(bytes32 _operation) external { uint ownerIndex =  
m_ownerIndex[uint(msg.sender)]; var pending = m_pending[_operation]; if  
(pending.ownersDone & ownerIndex > 0) { pending.yetNeeded++;  
pending.ownersDone -= ownerIndex; Revoke(msg.sender, _operation); } }
```

#### Explanation

The `revoke` function directly accesses the `\_operation` parameter without validation or sanitization, allowing an attacker to manipulate the operation ID to revoke a different owner's confirmation.

#### Impact

Unauthorized access to data and potential privilege escalation

#### Entry point

WalletAbi.revoke(bytes32 \_operation)

#### Parameters

\_operation

#### Exploitation steps

- Send a revoke request for an operation ID that is not owned by the current sender.
- Observe the revocation of another owner's confirmation.

#### Example payloads

- Revoke a confirmation for operation 'foo' owned by user2

#### Conditions

Access to operations owned by other users

#### Remediation

Validate and sanitize the `\_operation` parameter using a whitelist or other security measure.

## Secure example

```
function revoke(bytes32 _operation) external { require(_operation in approvedOperations); // validated operation ID }
```

Source lines: 1-286

### ► Finding 4.1: Vulnerability found (High)

#### Vulnerable code

```
function revoke(bytes32 _operation) external { uint ownerIndex = m_ownerIndex[uint(msg.sender)]; // make sure they're an owner if (ownerIndex == 0) return; uint ownerIndexBit = 2**ownerIndex; var pending = m_pending[_operation]; if (pending.ownersDone & ownerIndexBit > 0) { pending.yetNeeded++; pending.ownersDone -= ownerIndexBit; Revoke(msg.sender, _operation); } }
```

#### Explanation

Direct object reference vulnerability due to hardcoding the `msg.sender` value in the `\_ownerIndex` calculation without proper validation or access control checks.

#### Impact

Unauthorized access to wallet functionality and potential privilege escalation.

#### Entry point

revoke(bytes32 \_operation) external

#### Execution path

```
'''
1. revoke(bytes32 _operation) external
  - gets called when no other function matches
  - uses m_ownerIndex[uint(msg.sender)] to determine owner index
  - if ownerIndex == 0, returns without executing anything
  - otherwise, updates pending state and emits Revoke event
2. update pending state and emit Revoke event
'''
```

#### Parameters

\_operation, msg.sender

#### Exploitation steps

- Find a way to manipulate the `\_operation` parameter to bypass access control checks

#### Example payloads

- sha3('example\_operation')

**Conditions**

Owner is not authenticated or authorized.

**Remediation**

Use indirect references and proper access control checks to validate `msg.sender` and `\_operation` parameters.

**Source lines:** 96–414

► **Finding 4.1: Insecure Direct Object Reference (High)**

**Vulnerable code**

```
function revoke(bytes32 _operation) external {
    uint ownerIndex = m_ownerIndex[uint(msg.sender)];
    // ...
}
```

**Explanation**

The `\_ownerIndex` variable is directly computed from the message sender's address, making it vulnerable to an Insecure Direct Object Reference attack.

**Impact**

Unauthorized access to pending operations

**Entry point**

/revoke

**Execution path**

```
revoke(bytes32 _operation) -> calculate ownerIndex from msg.sender ->
revoke(msg.sender, _operation)
```

**Parameters**

msg.sender

**Exploitation steps**

- Get current owner index for msg.sender
- Use the owner index to access pending operations

**Conditions**

Attacker can access ownerIndex using msg.sender's address.

**Remediation**

Validate and sanitize user input, or use indirect references to access internal data.

## Secure example

```
uint ownerIndex = _ownerIndex(uint(msg.sender));  
if (ownerIndex > 0) {  
    // ...}
```

## File 5: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.654

Source lines: 1-39

### ► Finding 5.1: Insecure Direct Object Reference (Critical)

#### Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw)());
```

#### Explanation

The function call allows an attacker to manipulate the msg.sender, potentially bypassing access controls.

#### Impact

Unauthorized access to funds and potential privilege escalation.

#### Entry point

withdrawFunds(uint256 \_weiToWithdraw) public

#### HTTP methods

POST

#### Parameters

\_weiToWithdraw, msg.sender

#### Exploitation steps

- An attacker sends a withdrawal request with the attacker's address as msg.sender.
- The contract calls msg.sender.call.value(\_weiToWithdraw)().

#### Example payloads

- 100 ether

#### Conditions

An authenticated attacker can reach the withdrawFunds function.

#### Remediation

Use indirect references to external functions and validate user authorization.

## File 6: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.648

Source lines: 1-31

### ► Finding 6.1: Insecure Direct Object Reference (High)

#### Vulnerable code

```
receiver.transfer(amount);
```

#### Explanation

tx.origin is directly used without proper access control checks.

#### Impact

Unauthorized transfer of funds to malicious recipients.

#### Entry point

/transfer

#### HTTP methods

POST

#### Parameters

amount, receiver

#### Exploitation steps

- Send fake tx.origin from an intermediary contract

#### HTTP raw requests (Burp / ZAP)

```
POST /transfer HTTP/1.1
Host: 0x1234567890123456789012345678901234567890
Content-Length: 0
```

## File 7: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.645

Source lines: 1-85

### ► Finding 7.1: Insecure Direct Object Reference (Critical)

#### Vulnerable code

```
function ETH_VAULT(address _log) public { TransferLog = Log(_log); }
```

#### Explanation

Direct object reference vulnerability in the function `ETH\_VAULT` where `\_log` is not validated.

#### Impact

Unauthorized access to internal objects

#### Entry point

/transfer

#### HTTP methods

POST

#### Exploitation steps

- Attackers can execute arbitrary code by manipulating the `\_log` address.

#### Example payloads

- ``logAddress: 0x1234567890123456789012345678901234567890'``

#### HTTP raw requests (Burp / ZAP)

```
POST /transfer HTTP/1.1
Host: example.com
Content-Length: 70
```

```
logAddress: 0x1234567890123456789012345678901234567890'
```

## File 8: /work/project/dataset/vulnerable/ordering-attacks/eth\_tx\_order\_dependence\_minimal.sol

Similarity score: 0.609

Source lines: 1-42

### ► Finding 8.1: Vulnerability found (High)

#### Vulnerable code

```
require(msg.sender == owner); owner.transfer(reward);
```

#### Explanation

Direct object reference: msg.sender is used to access the owner's address without proper validation.

#### Impact

Privilege escalation and unauthorized data transfer

#### Entry point

/setReward()

#### HTTP methods

POST

#### Parameters

msg.value, reward

#### Exploitation steps

- Repeating the function with a crafted msg.value to bypass owner check

#### Example payloads

- large amount of Ether

#### Conditions

Authenticated sender can transfer reward.

#### Remediation

Use indirect references (e.g., keccak256(msg.sender)) and validate user authorization.

## File 9: /work/project/dataset/vulnerable/ordering-attacks/odds\_and\_evens.sol

Similarity score: 0.581

Source lines: 1-61

### ► Finding 9.1: Insecure Direct Object Reference (Critical)

#### Vulnerable code

```
players[tot] = Player(msg.sender, number);
```

#### Explanation

Direct reference to internal object 'players' without proper access controls.

#### Impact

Unauthorized access to data and potential privilege escalation.

#### Entry point

play(uint number) payable

#### Execution path

```
play -> players[tot] = Player(msg.sender, number); -> andTheWinnerIs() -> transfer
```

#### Parameters

number, msg.sender

#### Exploitation steps

- Attacker manipulates 'tot' to gain access to any player's data.

#### Conditions

Attacker can reach play function with a crafted 'number'.

#### Remediation

Validate and sanitize user input before accessing internal objects.

#### Secure example

```
players[tot] = Player(msg.sender, number); if (msg.value == 1 ether) { ... }
```

## **Assistant (triage / codebase)**

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

## Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)